

Initialization

Params: max generation G , population Size N , initial alpha population A_0 , $g = 1$, alpha pool $P = \{\}$

Alpha Population A_{g-1}

Step 1: Selection

Parent Alpha 1

$(-1 * \text{correlation}(\text{rank}(\text{volume}), \text{rank}(\text{open}), 10))$

Parent Alpha 2

$((-1 * \text{rank}(\text{stddev}(\text{high}, 10))) * \text{correlation}(\text{high}, \text{volume}, 10))$

Step 5: Population Update

False, $g += 1$,
Update to A_g

False, continue

$g \geq G$?

True

True

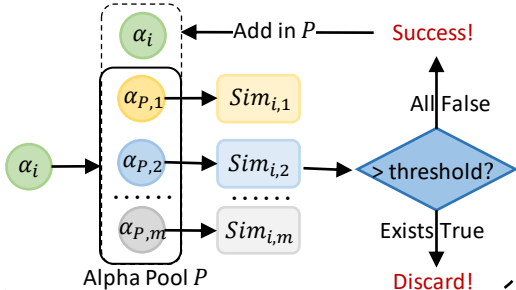
Generate N alphas?

Finish

Generated Alphas

```
{
  Generated Alpha 1:{
    "alpha":  $\text{ts\_min}((-1 * \text{rank}(\text{stddev}(\text{open}, 10))) * \text{correlation}(\text{rank}(\text{volume}), \text{high}, 10), 15)$ ,
    "code": "Calculate the ...",
    "fitness_value": 0.0383
  }
  .....
  Generated Alpha G*N: {
    "alpha": .....,
    "code": .....,
    "fitness_value": .....
  }
}
```

Step 4: Diversity Checker



Generated Alpha i

$\text{ts_min}((-1 * \text{rank}(\text{stddev}(\text{open}, 10))) * \text{correlation}(\text{rank}(\text{volume}), \text{high}, 10), 15)$

Fitness value: 0.0383

Step 3: Code Evaluator



Step 3.1: Code Generator

```
# Calculate the standard deviation of opening prices over a 10-day period
df['stddev_open_10'] = df['open'].rolling(window=10).std()
# Rank the standard deviation and multiply by -1
df['rank_stddev_open_10'] = df['stddev_open_10'].rank(axis=1, pct=True) * -1
df['rank_volume'] = df['volume'].rank(axis=1, pct=True)
df['correlation_rank_volume_high_10'] = df['rank_volume'].rolling(window=10).corr(df['high'])
# Calculate the time-series min over the past 15 days of the product of the previous factors
df['alpha'] = df['correlation_rank_volume_high_10'].rolling(window=15).min()
```



Step 3.2: Code Executor

Training Dataset

Code execution

Error Detected
Regenerate

Error Detect

No error

20-day Returns

Alpha Value

Fitness value: IC

Step 2: Evolutionary CoT



Step 2.1: Analysis

The Alpha 1 aims to capture the inverse relationship between the rank of the opening prices and the rank of the trading volume over the past 10 days...
The Alpha 2 attempts to exploit the relationship between price volatility and trading volume to generate predictive insights for stock returns...



Step 2.2: Crossover

In the crossover alpha, I exchanged elements from the original alphas to create a new combination... This new alpha aims to capture the relationship between the rank of trading volume and the standard deviation of opening prices, as well as their correlation with high prices... Therefore the **Crossover Alpha** is:
 $((-1 * \text{rank}(\text{stddev}(\text{open}, 10))) * \text{correlation}(\text{rank}(\text{volume}), \text{high}, 10))$



Step 2.3: Mutation

In the mutate alpha, I introduced a mutation by adding the time-series minimum operator (ts_min) to the crossover alpha... This introduces a new computational element into the alpha, aiming to capture the minimum value of the expression over the past 15 days... Therefore the **Mutation Alpha** is:
 $\text{ts_min}((-1 * \text{rank}(\text{stddev}(\text{open}, 10))) * \text{correlation}(\text{rank}(\text{volume}), \text{high}, 10), 15)$